

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
Санкт-Петербургский национальный исследовательский университет ИТМО
Мегафакультет трансляционных информационных технологий

Факультет прикладной информатики

Лабораторная работа №5

По дисциплине «Проектирование и реализация баз данных»
Вариант №19

Выполнил студент группы №К3240,
Гаенко Анастасия Константиновна

Преподаватель:
Говорова Марина Михайловна



Санкт-Петербург
2026

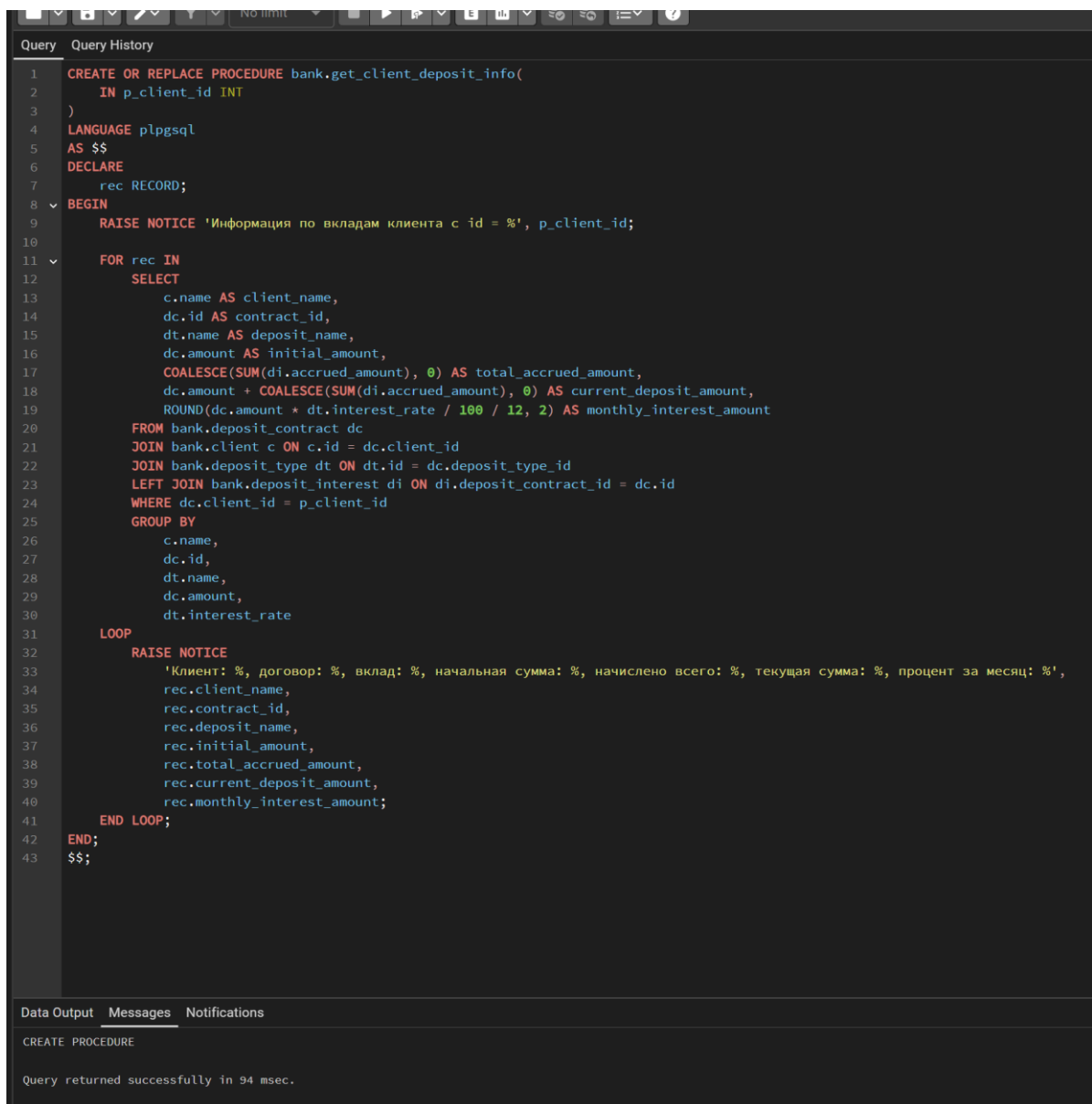
Цель работы

Изучить возможности СУБД PostgreSQL по созданию и использованию хранимых процедур, функций и триггеров, а также приобрести практические навыки автоматизации обработки данных и обеспечения целостности информации на примере базы данных «Банк».

Выполнение:

1 Процедура

о текущей сумме вклада и сумме начисленного за месяц процента для заданного клиента;



```
1 CREATE OR REPLACE PROCEDURE bank.get_client_deposit_info(
2     IN p_client_id INT
3 )
4 LANGUAGE plpgsql
5 AS $$
6 DECLARE
7     rec RECORD;
8 BEGIN
9     RAISE NOTICE 'Информация по вкладам клиента с id = %', p_client_id;
10
11     FOR rec IN
12         SELECT
13             c.name AS client_name,
14             dc.id AS contract_id,
15             dt.name AS deposit_name,
16             dc.amount AS initial_amount,
17             COALESCE(SUM(di.accrued_amount), 0) AS total_accrued_amount,
18             dc.amount + COALESCE(SUM(di.accrued_amount), 0) AS current_deposit_amount,
19             ROUND(dc.amount * dt.interest_rate / 100 / 12, 2) AS monthly_interest_amount
20         FROM bank.deposit_contract dc
21         JOIN bank.client c ON c.id = dc.client_id
22         JOIN bank.deposit_type dt ON dt.id = dc.deposit_type_id
23         LEFT JOIN bank.deposit_interest di ON di.deposit_contract_id = dc.id
24         WHERE dc.client_id = p_client_id
25         GROUP BY
26             c.name,
27             dc.id,
28             dt.name,
29             dc.amount,
30             dt.interest_rate
31     LOOP
32         RAISE NOTICE
33             'Клиент: %, договор: %, вклад: %, начальная сумма: %, начислено всего: %, текущая сумма: %, процент за месяц: %',
34             rec.client_name,
35             rec.contract_id,
36             rec.deposit_name,
37             rec.initial_amount,
38             rec.total_accrued_amount,
39             rec.current_deposit_amount,
40             rec.monthly_interest_amount;
41     END LOOP;
42 END;
43 $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 94 msec.

CALL bank.get_client_deposit_info(1);

```
Data Output  Messages  Notifications
NOTICE:  Информация по вкладам клиента с id = 1
NOTICE:  Клиент: Иванов Иван Иванович, договор: 1, вклад: Накопительный, начальная сумма: 120000.00, начислено всего: 1700.00, текущая сумма: 121700.00, процент за месяц: 850.00
NOTICE:  Клиент: Иванов Иван Иванович, договор: 4, вклад: Накопительный, начальная сумма: 200000.00, начислено всего: 0, текущая сумма: 200000.00, процент за месяц: 1416.67
NOTICE:  Клиент: Иванов Иван Иванович, договор: 5, вклад: Накопительный, начальная сумма: 2900000.00, начислено всего: 0, текущая сумма: 2900000.00, процент за месяц: 20541.67
NOTICE:  Клиент: Иванов Иван Иванович, договор: 8, вклад: Накопительный, начальная сумма: 1000000.00, начислено всего: 0, текущая сумма: 1000000.00, процент за месяц: 7083.33
NOTICE:  Клиент: Иванов Иван Иванович, договор: 11, вклад: Накопительный, начальная сумма: 3000000.00, начислено всего: 100000.00, текущая сумма: 3100000.00, процент за месяц: 21250.00
CALL

Query returned successfully in 69 msec.
```

2 Процедура

Добавить данные о новом вкладе клиента;

```

CREATE OR REPLACE PROCEDURE bank.add_client_deposit(
    IN p_date_start DATE,
    IN p_date_end DATE,
    IN p_amount NUMERIC,
    IN p_currency_id INT,
    IN p_deposit_type_id INT,
    IN p_employee_id INT,
    IN p_client_id INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_new_id INT;
BEGIN
    SELECT COALESCE(MAX(id), 0) + 1
    INTO v_new_id
    FROM bank.deposit_contract;

    INSERT INTO bank.deposit_contract (
        id,
        date_start,
        date_end,
        amount,
        currency_id,
        deposit_type_id,
        employee_id,
        client_id
    )
    VALUES (
        v_new_id,
        p_date_start,
        p_date_end,
        p_amount,
        p_currency_id,
        p_deposit_type_id,
        p_employee_id,
        p_client_id
    );

    RAISE NOTICE 'Добавлен новый вклад. Номер договора: %', v_new_id;
END;
$$;

```

```
CALL bank.add_client_deposit(
    '2026-06-01',
    '2027-06-01',
    100000.00,
    1,
    1,
    1,
    2
);

SELECT *
FROM bank.deposit_contract
ORDER BY id;
```

Output Messages Notifications

id [PK] integer	date_start date	date_end date	amount numeric (12,2)	currency_id integer	deposit_type_id integer	employee_id integer	client_id integer
1	2026-04-01	2026-10-01	120000.00	1	1	1	1
2	2026-03-15	2027-03-15	300000.00	1	2	1	3
3	2026-04-05	2026-07-05	15000.00	1	3	3	5
4	2026-05-01	2026-11-01	200000.00	1	1	1	1
5	2026-05-06	2027-11-01	2900000.00	1	1	1	1
6	2024-05-12	2028-11-11	3000000.00	4	1	1	2
7	2024-05-12	2028-11-11	3000000.00	2	1	1	2
8	2025-04-23	2029-03-24	1000000.00	1	1	1	1
9	2025-12-23	2029-11-24	3000000.00	1	1	1	2
10	2025-03-23	2029-10-24	13000000.00	1	1	1	3
11	2024-05-12	2026-04-28	3000000.00	4	1	1	1
12	2026-06-01	2027-06-01	100000.00	1	1	1	2

3 Процедура

найти клиентов банка, не имеющих задолженности по кредитам.

```

1 CREATE OR REPLACE PROCEDURE bank.get_clients_without_credit_debt()
2 LANGUAGE plpgsql
3 AS $$
4 DECLARE
5     rec RECORD;
6 BEGIN
7     RAISE NOTICE 'Клиенты без задолженности по кредитам:>';
8
9     FOR rec IN
10        SELECT
11            c.id,
12            c.name,
13            c.phone_number,
14            c.email
15        FROM bank.client c
16        WHERE NOT EXISTS (
17            SELECT 1
18            FROM bank.credit_contract cc
19            JOIN bank.payment_schedule ps ON ps.contract_id = cc.id
20            JOIN bank.payment_fact pf ON pf.payment_id = ps.id
21            WHERE cc.client_id = c.id
22                  AND pf.is_overdue = TRUE
23        )
24        ORDER BY c.id
25    LOOP
26        RAISE NOTICE
27            'id: %, ФИО: %, телефон: %, email: %',
28            rec.id,
29            rec.name,
30            rec.phone_number,
31            rec.email;
32    END LOOP;
33 END;
34 $$;
35
36 CALL bank.get_clients_without_credit_debt();

```

Data Output Messages Notifications

```

NOTICE:  Клиенты без задолженности по кредитам:
NOTICE:  id: 1, ФИО: Иванов Иван Иванович, телефон: +79990000001, email: ivanov1@mail.ru
NOTICE:  id: 2, ФИО: Петров Петр Петрович, телефон: +79990000002, email: petrov2@mail.ru
NOTICE:  id: 3, ФИО: Сидорова Анна Викторовна, телефон: +79990000003, email: sidorova3@mail.ru
NOTICE:  id: 4, ФИО: Кузнецов Алексей Сергеевич, телефон: +79990000004, email: kuznetsov4@mail.ru
NOTICE:  id: 5, ФИО: Смирнова Мария Олеговна, телефон: +79990000005, email: smirnova5@mail.ru
CALL

```

Триггеры

1. Проверка минимальной суммы вклада

```

1 CREATE OR REPLACE FUNCTION bank.check_deposit_min_amount()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6     v_min_amount NUMERIC(12,2);
7 BEGIN
8     SELECT minimum_amount
9     INTO v_min_amount
10    FROM bank.deposit_type
11   WHERE id = NEW.deposit_type_id;
12
13     IF NEW.amount < v_min_amount THEN
14         RAISE EXCEPTION
15             'Сумма вклада % меньше минимальной суммы %',
16             NEW.amount, v_min_amount;
17     END IF;
18
19     RETURN NEW;
20 END;
21 $$;
22
23 CREATE TRIGGER trg_check_deposit_min_amount
24 BEFORE INSERT OR UPDATE ON bank.deposit_contract
25 FOR EACH ROW
26 EXECUTE FUNCTION bank.check_deposit_min_amount();

```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 38 msec.

2. Проверка минимального срока вклада

```

1 CREATE OR REPLACE FUNCTION bank.check_deposit_min_term()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6     v_min_term INT;
7     v_term_months INT;
8 BEGIN
9     SELECT minimum_term
10    INTO v_min_term
11   FROM bank.deposit_type
12  WHERE id = NEW.deposit_type_id;
13
14     v_term_months :=
15         EXTRACT(YEAR FROM age(NEW.date_end, NEW.date_start)) * 12
16         + EXTRACT(MONTH FROM age(NEW.date_end, NEW.date_start));
17
18     IF v_term_months < v_min_term THEN
19         RAISE EXCEPTION
20             'Срок вклада % мес. меньше минимального срока % мес.',
21             v_term_months, v_min_term;
22     END IF;
23
24     RETURN NEW;
25 END;
26 $$;
27
28 CREATE TRIGGER trg_check_deposit_min_term
29 BEFORE INSERT OR UPDATE ON bank.deposit_contract
30 FOR EACH ROW
31 EXECUTE FUNCTION bank.check_deposit_min_term();

```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 43 msec.

3. Проверка минимальной суммы кредита


```

1  CREATE OR REPLACE FUNCTION bank.check_credit_min_amount()
2  RETURNS TRIGGER
3  LANGUAGE plpgsql
4  AS $$
5  DECLARE
6      v_min_amount NUMERIC(12,2);
7  BEGIN
8      SELECT minimum_amount
9      INTO v_min_amount
10     FROM bank.credit_type
11     WHERE id = NEW.credit_type_id;
12
13     IF NEW.amount < v_min_amount THEN
14         RAISE EXCEPTION
15             'Сумма кредита % меньше минимальной суммы %',
16             NEW.amount, v_min_amount;
17     END IF;
18
19     RETURN NEW;
20 END;
21 $$;
22
23 CREATE TRIGGER trg_check_credit_min_amount
24 BEFORE INSERT OR UPDATE ON bank.credit_contract
25 FOR EACH ROW
26 EXECUTE FUNCTION bank.check_credit_min_amount();

```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 39 msec.

4. Проверка минимального срока кредита

```

1 CREATE OR REPLACE FUNCTION bank.check_credit_min_term()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6     v_min_term INT;
7     v_term_months INT;
8 BEGIN
9     SELECT minimum_term
10    INTO v_min_term
11   FROM bank.credit_type
12  WHERE id = NEW.credit_type_id;
13
14     v_term_months :=
15         EXTRACT(YEAR FROM age(NEW.date_end, NEW.date_start)) * 12
16         + EXTRACT(MONTH FROM age(NEW.date_end, NEW.date_start));
17
18     IF v_term_months < v_min_term THEN
19         RAISE EXCEPTION
20             'Срок кредита % мес. меньше минимального срока % мес.',
21             v_term_months, v_min_term;
22     END IF;
23
24     RETURN NEW;
25 END;
26 $$;
27
28 CREATE TRIGGER trg_check_credit_min_term
29 BEFORE INSERT OR UPDATE ON bank.credit_contract
30 FOR EACH ROW
31 EXECUTE FUNCTION bank.check_credit_min_term();

```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 38 msec.

5. Запрет платежей 29, 30 и 31 числа

```

1 CREATE OR REPLACE FUNCTION bank.check_payment_schedule_day()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     IF EXTRACT(DAY FROM NEW.payment_date) IN (29, 30, 31) THEN
7         RAISE EXCEPTION
8             'Нельзя указывать день выплаты 29, 30 или 31. Указанная дата: %',
9             NEW.payment_date;
10    END IF;
11
12    RETURN NEW;
13 END;
14 $$;
15
16 CREATE TRIGGER trg_check_payment_schedule_day
17 BEFORE INSERT OR UPDATE ON bank.payment_schedule
18 FOR EACH ROW
19 EXECUTE FUNCTION bank.check_payment_schedule_day();

```

6. Автоматическое определение просрочки платежа

```

1 CREATE OR REPLACE FUNCTION bank.set_payment_overdue_status()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6     v_plan_date DATE;
7 BEGIN
8     SELECT payment_date
9     INTO v_plan_date
10    FROM bank.payment_schedule
11   WHERE id = NEW.payment_id;
12
13     IF NEW.payment_date IS NULL THEN
14         NEW.is_overdue := TRUE;
15     ELSIF NEW.payment_date > v_plan_date THEN
16         NEW.is_overdue := TRUE;
17     ELSE
18         NEW.is_overdue := FALSE;
19     END IF;
20
21     RETURN NEW;
22 END;
23 $$;
24
25 CREATE TRIGGER trg_set_payment_overdue_status
26 BEFORE INSERT OR UPDATE ON bank.payment_fact
27 FOR EACH ROW
28 EXECUTE FUNCTION bank.set_payment_overdue_status();

```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 36 msec.

7. Логирование действий с договорами

```

1 CREATE OR REPLACE FUNCTION bank.log_contract_action()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     IF TG_TABLE_NAME = 'deposit_contract' THEN
7         INSERT INTO bank.action_log (
8             table_name,
9             operation_type,
10            description
11        )
12        VALUES (
13            TG_TABLE_NAME,
14            TG_OP,
15            'Операция ' || TG_OP ||
16            ' по договору вклада №' || NEW.id ||
17            ', клиент №' || NEW.client_id ||
18            ', сумма ' || NEW.amount
19        );
20
21        RETURN NEW;
22
23     ELSIF TG_TABLE_NAME = 'credit_contract' THEN
24         INSERT INTO bank.action_log (
25             table_name,
26             operation_type,
27             description
28         )
29         VALUES (
30             TG_TABLE_NAME,
31             TG_OP,
32             'Операция ' || TG_OP ||
33             ' по кредитному договору №' || NEW.id ||
34             ', клиент №' || NEW.client_id ||
35             ', сумма ' || NEW.amount
36         );
37
38        RETURN NEW;
39    END IF;
40
41    RETURN NEW;
42 END;
43 $$;
44
45 CREATE TRIGGER trg_log_deposit_contract
46 AFTER INSERT OR UPDATE ON bank.deposit_contract
47 FOR EACH ROW
48 EXECUTE FUNCTION bank.log_contract_action();
49
50 CREATE TRIGGER trg_log_credit_contract
51 AFTER INSERT OR UPDATE ON bank.credit_contract
52 FOR EACH ROW
53 EXECUTE FUNCTION bank.log_contract_action();

```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 43 msec.

Вывод

В ходе выполнения лабораторной работы были изучены механизмы создания и использования хранимых процедур, функций и триггеров в PostgreSQL. Для базы данных «Банк» были разработаны процедуры для получения информации о текущем состоянии вкладов клиентов, добавления новых вкладов и поиска клиентов, не имеющих задолженности по кредитам. Также были созданы триггеры, обеспечивающие контроль корректности вводимых данных и автоматизацию отдельных операций в базе данных.

В результате работы были закреплены навыки разработки серверной логики средствами PostgreSQL, а также изучены способы обеспечения

целостности и согласованности данных на уровне СУБД. Использование процедур и триггеров позволило сократить объём ручной обработки данных и повысить надёжность работы информационной системы.